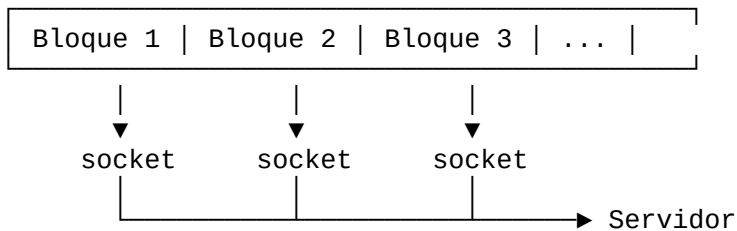


Ejercicio 3 — Transferencia de un archivo mediante sockets

Modificar el programa cliente-servidor desarrollado en la práctica anterior para permitir que el cliente envíe un archivo al servidor utilizando un socket TCP. En la práctica anterior, el cliente y el servidor intercambiaron mensajes de texto. Ahora utilizaremos la misma idea para transferir un archivo. La diferencia principal es que un archivo puede ser mucho más grande que un mensaje. Por esta razón, no debemos intentar enviar todo el archivo de una sola vez.

En su lugar, el archivo se divide en pequeños bloques:

Archivo



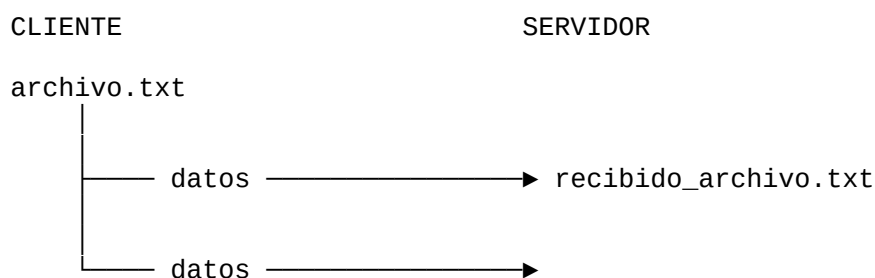
El servidor recibe los bloques y los va escribiendo en un nuevo archivo. El cliente tendrá un archivo llamado, por ejemplo:

`archivo.txt`

El cliente se conectará al servidor y enviará el archivo. El servidor lo recibirá y creará:

`recibido_archivo.txt`

Al final tendremos:



La comunicación será la siguiente:

1. El servidor inicia y espera una conexión.
2. El cliente se conecta.
3. El cliente indica el nombre del archivo.
4. El cliente abre el archivo.
5. El cliente lee un bloque.
6. El cliente envía el bloque.

7. El servidor recibe el bloque.
8. El servidor escribe el bloque en un archivo.
9. Se repiten los pasos 5-8 hasta terminar el archivo.
10. El servidor informa que la transferencia terminó.
11. Cliente y servidor cierran la conexión.

Ejercicio 1 — Mostrar el tamaño del archivo

Modifica el cliente para mostrar el tamaño del archivo antes de enviarlo.

Ejemplo:

```
Archivo: archivo.txt
Tamaño: 2450 bytes
```

```
Enviando archivo...
Archivo enviado correctamente.
```

Ejercicio 2 — Contar los bytes enviados

Modifica el cliente para llevar un contador.

Por ejemplo:

```
Enviando archivo...
```

```
Bytes enviados: 1024
Bytes enviados: 2048
Bytes enviados: 3072
```

```
Archivo enviado correctamente.
Total enviado: 3500 bytes
```

Ejercicio 3 — Mostrar los bytes recibidos

Haz lo mismo en el servidor.

El servidor deberá mostrar:

```
Cliente conectado
Archivo recibido: archivo.txt
```

```
Bytes recibidos: 1024
Bytes recibidos: 2048
Bytes recibidos: 3072
```

```
Archivo recibido correctamente.
Total recibido: 3500 bytes
```

Ejercicio 4 — Transferir una imagen

Después de comprobar que funciona con un archivo de texto, intenta transferir una imagen.

Por ejemplo:

foto.jpg

La carpeta podría quedar:

```
practica_archivos/  
├── servidor.py  
├── cliente.py  
├── foto.jpg  
└── recibido_foto.jpg
```

Abre ambos archivos y verifica que la imagen recibida pueda abrirse correctamente.

1. ¿Cuál es la diferencia entre enviar un mensaje de texto y enviar un archivo?
2. ¿Por qué dividimos el archivo en bloques?
3. ¿Qué significa `rb`?
4. ¿Qué significa `wb`?
5. ¿Qué sucede cuando `recv()` devuelve una cantidad vacía de datos?
6. ¿Por qué el servidor necesita escribir los datos recibidos en un archivo?
7. ¿Qué ocurriría si utilizáramos un bloque de 10 bytes en lugar de 1024?
8. ¿Qué ventajas tiene utilizar bloques para transferir archivos grandes?
9. ¿Qué ocurre si el archivo original tiene una imagen?
10. ¿Cómo podríamos modificar el programa para que el servidor también pudiera enviar archivos al cliente?

Entregable

Cada estudiante deberá entregar:

Archivos

```
servidor.py  
cliente.py
```

Evidencias

Una captura donde se observe:

- El cliente enviando el archivo.
- El servidor recibiendo el archivo.
- El archivo original y el archivo recibido.

Reporte

Incluir las respuestas a las preguntas de reflexión.